EPSR-test-ex75 sdfcube water-water (identical), chained 6 times

Program(s): sdfcube | chained passes: 6 | verification: ALL IDENTICAL (all 6 passes)

What this case tests

Exercises `sdfcube` [1], which computes the spatial density function of one molecule type around another directly on a 3D grid for visualisation, including accumulation of the cube over a chained run.

Code exercised

Fortran (original, fortran_src/):

- `sdfcube.f`: `run_sdfcube`, `sdfcube` (grid histogram), `initcube`, `outcube` (cube file, accumulation)
- `sharm_routines_rev1.f`: `form_eulerangles`; `auxiliary_routines.f`: `rotateeuler`; `process_symbols.f`: `check_symbol`
- `setup_symbols_generic.f`: `setup_symbols_sdfcube`

Rust (port, rust/epsr-core/src/):

- `tools/sdfcube.rs`: `run`, `sdfcube`, `initcube`, `outcube`
- `tools/sharm_routines.rs`: `form_eulerangles`, `rotateeuler`, `write_mol_xyz`; `tools/sharm.rs`: `read_axis`, `average_molecule`
- `symtab_tools.rs`: `SDFCUBE` keyword table

Physical system

The EPSR-test configuration `200DMSO400Water.ato`: 200 DMSO (atoms S, C, C, O, 6 H) and 400 water molecules (O1, H1, H1; SPC/E parameters [2]), 3200 atoms at 300 K in a cubic box of side 33.12 Å, a model of the DMSO-water mixtures of Soper and Luzar [3]. `sdfcube` histograms the centres of the second molecules, rotated into the frame of each central molecule (Euler angles from the axis definitions), on a cubic grid and writes the spatial density function as a Gaussian `.cube` file together with the averaged molecules (`-first.xyz`, `-second.xyz`). Here water around water (centre = mass centre of O1/H1, z axis to the H-H midpoint, y from H 2) on a 17×17×17 grid (`mpixel 8`, `rmax 6`: 0.75 Å steps, stored in bohr in the cube header).

Inputs

Input files written by hand from the keyword tables (`setup_symbols_*`), one `keyword value` per line, ended by `q`; `200DMSO400Water.ato` copied from EPSR-test. `water.SDFCUBE.dat` with `nsumt 0`, `averagemol y`.

How the accuracy was verified

`commands.txt` (1 program call, repeated 6 times, each pass reading the files written by the previous one) was run with the Fortran reference build (`rust/tools/build_fortran_reference.sh`: the original sources, gfortran -O2 -fno-tree-vectorize, single thread) and with the Rust `epsr-rs` on identical copies of the inputs, and every file written was compared byte for byte (`rust/tools/run_ex_test.sh`, `compare_outputs.py`; no masking except where noted below). The Fortran output is stored in `fortran_reference.tar.gz`. Result of the final regression run of the complete port: every file identical in all 6 passes.
Six passes. Pass 1 creates `water.SDFCUBE.cube`; pass 6 (`nsumt 5`) reads the step size and the old cube (`initcube`, `outcube`) and forms `fc1*old` + `fc*new`; final `nsumt 6`. The cube, both xyz files and the updated input file are compared after each pass. No `sharm`/`sdf` patch is involved.

Notes

Reproduced Fortran behaviour: `fc1` is computed from an uninitialised variable that gfortran -O2 folds to 1.0 (checked in the reference object code); for identical molecules `averagemol` is read from the `axiss1` value (missing `iref=iref+2`), so a single molecule is written to the xyz files.

References

[1] A. K. Soper, "Empirical Potential Structure Refinement - EPSRshell: a user's guide", Technical Report RAL-TR-2011-012, STFC Rutherford Appleton Laboratory (2011).
[2] H. J. C. Berendsen, J. R. Grigera and T. P. Straatsma, "The missing term in effective pair potentials", J. Phys. Chem. 91, 6269-6271 (1987).
[3] A. K. Soper and A. Luzar, "A neutron diffraction study of dimethyl sulphoxide-water mixtures", J. Chem. Phys. 97, 1320-1331 (1992).
[4] GNU Fortran (gfortran) 15.2, GCC, https://gcc.gnu.org/fortran/ - reference build with -O2 -fno-tree-vectorize.